

长周期智能体的环境工程：从任务时长到可验证的因果工作

Rayan Garg (演讲) Codex (整理)

2026 年 8 月 5 日



演讲者：Rayan Garg (Theta Software)

视频作者/频道：AI Engineer

发布日期：2026 年 8 月 1 日

视频时长：00:21:14

视频链接：<https://www.youtube.com/watch?v=2aS7aKoXn64>

PDF 制作 Skill：<https://github.com/wdkns/wdkns-skills>

目录

1 先定义测量对象：长周期是一组坐标，不是一个时长标签	2
1.1 移动的能力前沿：用连续尺度比较任务	2
1.2 人类时域与成功阈值：数字必须带完整限定	2
1.3 模型原生负担：过程轨迹中的四种观察量	3
1.4 两类测量噪声：系统差异与人类差异	4
1.5 本章小结	5
2 让早期决策产生后果：从工具数量走向因果深度	5
2.1 工具协调只是入口	5
2.2 状态变化：让步骤彼此约束	5
2.3 可并行复杂度与顺序复杂度	6
2.4 任务歧义：让探索成为能力的一部分	6
2.5 本章小结	7
3 验证结果，也验证路径：开放任务的双重证据	7
3.1 从硬可验证领域走向开放输出	7
3.2 最终状态与过程轨迹：结果证据和手段证据	8
3.3 奖励劫持：表面成功为何可能失真	9
3.4 参考答案的有限角色：验证约束，保留路径多样性	10
3.5 本章小结	10
4 把评审者也当作智能体：工具复用、权限收缩与可查询轨迹	10
4.1 “评审者也是智能体”意味着什么	10
4.2 部署失败案例：从行动链到验证链	11
4.3 权限不对称：共享观察接口，收回变更能力	11
4.4 把长轨迹加工成可查询证据	12
4.5 本章小结	13
5 让反馈可学习、让基准可信：量规工程与四维质量检查	13
5.1 可学习性与奖励信号密度	14
5.2 混合验证与条件性部分得分	14
5.3 量规质量保障：先检查评分规则	15
5.4 基准的四维质量检查	16
5.5 讲者的收束案例及其证据边界	17
5.6 本章小结	18
6 总结与延伸：用闭环审查长周期环境	18
6.1 讲者的收束判断：先检查数据能测量什么	18
6.2 跨节综合：六个环节怎样互相约束	18
6.3 面向实践的七步审查链	19
6.4 局限与开放问题	19
6.5 本章小结	20

1 先定义测量对象：长周期是一组坐标，不是一个时长标签

讨论长周期智能体之前，需要先回答一个更基础的问题：这里的“长”究竟测量什么。来源把长周期任务（long-horizon task）放在持续移动的能力前沿上。随着模型与运行系统进步，去年需要长时间自主工作的任务，今年可能已经变成常规任务。因此，“长周期”适合表达相对位置，无法充当永久不变的类别标签（00:48--01:50）。

1.1 移动的能力前沿：用连续尺度比较任务

来源将长周期程度称为标量指标（scalar metric）：任务可以在一条连续尺度上相互比较，一个任务比另一个任务更长、更依赖持续协调；“长”与“短”的分界则会随能力前沿移动。这个表述包含两个重要前提。

第一，比较必须绑定具体时间、模型和评估设置。脱离这些上下文，一句“模型已经能做十六小时任务”无法说明该数字来自哪类任务，也无法说明成功的判定标准。第二，墙钟运行时间只描述进程持续多久；它没有直接说明任务对记忆、规划、工具调用和环境状态提出了什么要求。

核心判断

长周期能力是一组带协议上下文的坐标。任何单独的小时数、token 数或步骤数，都只能描述一个侧面；可信比较还需要说明成功阈值、任务分布、运行框架与测量方法。

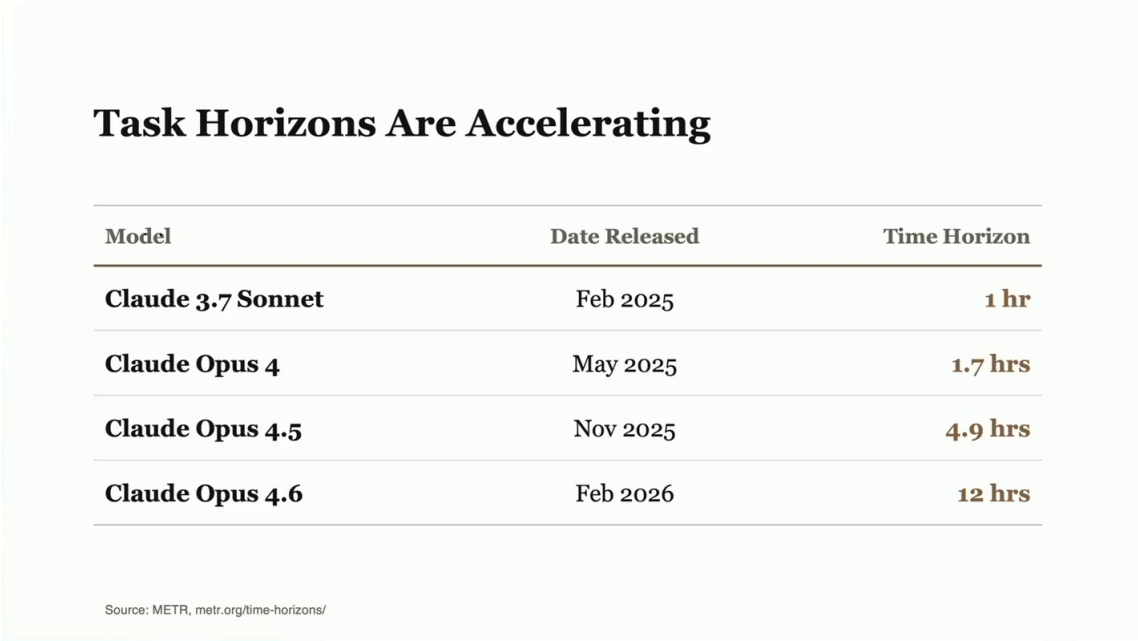


图 1：任务时域随模型代际快速扩展；同一个“长周期”标签会随着能力前沿移动。¹

1.2 人类时域与成功阈值：数字必须带完整限定

任务时域（time horizon）以合格人类完成任务所需时间标定模型可以自主完成的任务跨度，并同时绑定一个成功概率。与它相关的人类时域（human horizon）提供了直观的劳动参照：读者容易

¹ 视频画面时间区间：00:48–01:13。

理解“这项工作通常需要一名合格人员花费多少时间”。

来源图表给出的示例是：在该 METR 任务集和测量协议下，Claude Opus 4.6 在 50% 成功率阈值处对应的预测人类时域为 12 小时（01:52--02:29）。“12 小时”和“50%”必须成对出现。该示例说明指标结构，无法脱离任务分布、人员选择和估时方法推广为模型的一般能力上限。来源同时说明，人类耗时的测量具有严格方法，演讲没有展开其细节。

数字的证据边界

已批准画面确认 50% 阈值与 12 小时预测值，并明示当前任务集对 16 小时以上的测量不可靠。因此，12 小时是绑定该模型、阈值与任务集的来源图表值，无法写成跨版本通用结论。

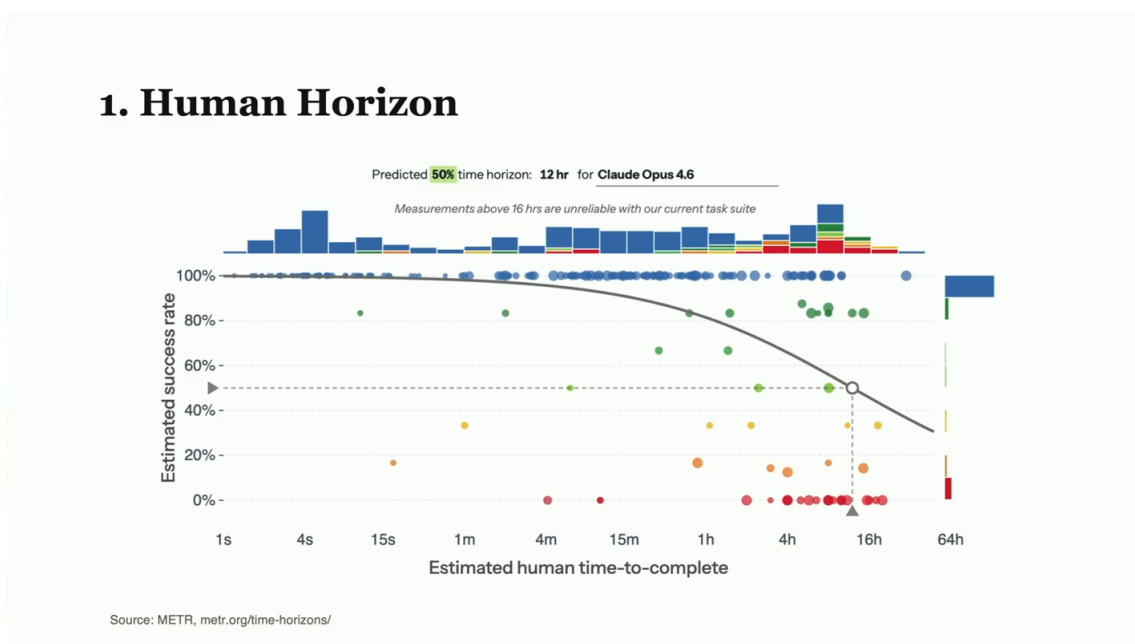


图 2：人类时域把成功率阈值与人类完成时间绑定；阈值、模型和任务集可靠范围必须一起解释。²

1.3 模型原生负担：过程轨迹中的四种观察量

人类时间提供外部参照，模型原生轨迹指标（model-native trajectory measure）则从智能体自身的执行过程观察负担。来源列出 token、步骤、工具调用和轨迹长度。四者相互关联，却各自回答不同问题：

- **token** 反映模型读取与生成的信息规模；
- **步骤**反映任务被拆成多少次连续决策；
- **工具调用**反映模型与外部系统交互的次数及类型；
- **过程轨迹（trajectory）**记录步骤、观察、调用和状态变化组成的完整序列。

轨迹变长之后，智能体需要在越来越多的历史信息中维持目标与约束。来源特别提到，上下文窗口与上下文压缩（context compaction）会影响这种持续能力（03:23--04:05）。更大的窗口可以

²视频画面时间区间：01:52--02:29。

容纳更多原始信息；上下文压缩会整理既有内容以延续工作，同时带来遗漏关键细节的风险。这两个机制需要分别讨论，不能合并成一个模糊的“记忆能力”。

字幕还出现了约五十万 token 和“百万级轨迹”的说明（03:11--03:57），相关模型版本、单位及经验性质没有在当前文字证据中得到可靠确认。本节因此保留机制结论，不把这些数量当作已验证实验结果。

2. Model Horizon

- Measuring tasks in “model units”: tokens, steps, tool calls, etc.
- Can be much more noisy, especially across different harnesses and models
 - Some models are more token-efficient than others on the same tasks (e.g. GPT-5.5 vs Opus 4.8)
 - Easiest to interpret when holding these variables constant: if a task takes more tokens, probably harder / longer-horizon
- It’s an imperfect metric, but still an important one to help define technical frontiers for models: what makes a task difficult for models (e.g. compaction, memory, coherence over long trajectories, etc.)

图 3: 模型原生时域直接暴露轨迹负担，同时强烈依赖模型与运行框架。³

1.4 两类测量噪声：系统差异与人类差异

模型原生指标的主要噪声来自系统侧。相同任务在不同模型、不同运行框架（harness）或不同上下文管理策略下，可能消耗完全不同的 token 和步骤。运行框架包含工具接入、上下文管理、生命周期和观测接口；框架差异会改变轨迹形态，因此原始 token 数不能直接当作跨系统能力排名。控制模型代际和框架后，同一系统内部的轨迹变化仍可用于诊断技术进步（02:46--04:18）。

人类时域的噪声来自另一组机制。来源用 Excel 批量修改格式举例：金融分析师可能需要数天手工调整大型工作簿的主题与颜色，智能体却可能编写一个简短的 Python 脚本快速完成同类重复操作（04:40--05:18）。人类耗时很长，可能反映手工流程的低效，未必表示任务要求深层推理。

估时还依赖执行者的专业熟练度。同一金融或编码任务由经验不同的人完成，平均耗时可能明显变化；当任务只由人群中的少数顶尖专家胜任时，合格样本更稀少，估时也更不稳定（05:22--06:08）。因此，人类时域强调劳动可解释性，模型原生轨迹指标揭示智能体的工程负担，二者形成互补视角。

Excel 与 Python 的类比

这组比较揭示“参照系错位”：人工逐格修改像逐行手工搬运数据，脚本则把重复规则交给解释器批量执行。人类用时衡量的是既定人类工作流，智能体可能选择另一条路径。评估设计需要检查两种工作流的瓶颈是否一致。

³视频画面时间区间：02:29–04:05。

1.5 本章小结

长周期首先是测量问题。标量指标说明能力边界会移动；任务时域与人类时域提供带成功阈值的劳动参照；模型原生轨迹指标揭示 token、步骤、工具调用和过程轨迹形成的执行压力。两组坐标各有噪声，也各有不可替代的信息。下一章据此把问题从“任务持续多久”推进到“环境是否让早期行动真实约束后续行动”。

2 让早期决策产生后果：从工具数量走向因果深度

测量坐标只能描述任务有多长，环境结构才决定这段长度是否承载真实能力要求。环境复杂度 (environment complexity) 关注智能体怎样协调工具、读取或改变状态，以及前序行动如何限制后续选择。工具更多可能增加操作量；只有持续的状态变化和依赖关系，才会把一串操作变成一条有因果连续性的工作链 (06:36--08:28)。

2.1 工具协调只是入口

工具协调 (tool coordination) 指跨多个工具或外部依赖传递信息、安排调用并汇总结果。来源先给出一个低复杂度起点：智能体只需读取代码库中的一个文件或一组文件。随后，任务扩展到日志观测、CI/CD、云监控以及数据库读写，信息需要在这些系统之间流动 (06:55--07:49)。

这里的重点是关系，而非产品清单。单独调用十个互不相关的工具，主要增加切换成本；从监控现象定位日志、从日志推断代码缺陷、再通过代码与部署状态验证修复，才形成跨工具的信息闭环。字幕中的具体产品名存在自动识别误差，因此正文保留工具类别，不锁定尚未由画面确认的品牌拼写。

环境复杂度的第一层

工具数量只是可见表象。真正需要测量的是信息是否必须跨工具传递、不同工具的结果是否共同约束下一步，以及任何一次调用是否会改变后续可观察状态。

2.2 状态变化：让步骤彼此约束

状态变化 (state change) 是行动对环境产生的持续影响：一次查询改变了后续判断依据，一次代码修改改变了构建输入，一次数据库写入改变了后续读取结果。它与无状态查询的区别在于，环境在行动前后已经不同，后续可选动作也随之改变 (07:49--08:33)。

来源用日志或仪表盘诊断说明这种约束。若智能体早期写错查询或误读日志，它可能把错误假设带入后续定位，随后修改错误组件，最后在错误路径上不断累积“看似合理”的证据 (08:53--09:10)。错误级联并非单纯多做几步；早期结果已经改变后续搜索空间，使后面的每一步都依赖一个有缺陷的前提。

常见设计误区

把互不相关的任务排成长队列只能人为增加总时长。来源的直接例子是串联独立任务；在没有画面证据时，不能把它改写成某种特定强制串行机制。环境是否具有深度，应由状态持续性和前后依赖证明。

Environment Complexity — State Changes

The degree to which the content of the environment changes throughout the task

TAKEAWAYS

- Tasks can be made artificially long horizon by chaining together unrelated independent tasks. However, merely making tasks take longer to complete misses the mark; a key component of adding complexity is that an agent's initial decisions influence its options for later decisions
- Sequential complexity: a bad early query or misread dashboard cascades into every downstream step, like a deploy and CI run that takes longer to resolve end-to-end
- Parallelizable complexity: Summarizing documentation in a large codebase can be made much quicker by reading each file independently

图 4: 环境复杂度的关键是早期决策改变后续选项，而非简单增加总时长。⁴

2.3 可并行复杂度与顺序复杂度

来源通过两个例子区分工作量与依赖深度。可并行复杂度 (parallelizable complexity) 描述可以拆成近似独立子任务的工作。例如，分析大型代码库时，多个子代理可以分别检查不同文件，再把发现汇总给协调者 (08:33--08:55)。任务规模很大，分支之间却很少互相改变状态。

顺序复杂度 (sequential complexity) 描述前序判断或状态变化会约束后序行动的依赖深度。日志诊断就具有这种性质：先定位哪个阶段失败，才能决定查看哪组指标；前一步误读会改变下一步查询，并可能最终导向错误修复 (08:53--09:10)。

这一区分可以用软件工程中的两种工作方式理解。独立静态检查像把一批文件分给多个审阅者，结果最后合并；线上故障排查像沿着一条证据链逐层缩小范围，每一层的结论都会决定下一层去哪里寻找。前者主要考验覆盖面与汇总能力，后者额外考验因果连续性和纠错能力。

两个概念的边界

可并行复杂度可以很高，例如需要检查数千个文件；顺序复杂度也可能出现在步骤不多的任务中，例如三次连续操作彼此改变数据库状态。任务深度取决于依赖结构，无法由步骤总数直接推出。

2.4 任务歧义：让探索成为能力的一部分

任务歧义 (ambiguity) 来自起始指令、工件或信息中的合理未定项。真实工作通常不会提供唯一且完备的操作清单；智能体需要阅读材料、形成假设、探索环境，再选择一条可行路径 (09:09--09:51)。这类歧义不同于漏写关键验收条件。前者保留合理探索空间，后者会让任务意图本身无法判定。

歧义提升了能力要求，也提升了评估难度。给定信息越开放，可能的正确路径越多；一个只匹配标准答案的评分器容易把有效替代方案判错 (09:51--10:06)。因此，任务设计者需要同时保存

⁴ 视频画面时间区间：07:49–09:10。

两类边界：一类是不容违反的目标与约束，另一类是允许智能体自行选择的策略空间。

任务深度的三项检查

判断一个任务是否真正具有长周期能力含义，可以依次检查：信息是否需要跨工具流动；行动是否造成持久状态变化；前序决策是否限制后序选择。任务歧义进一步检验智能体能否在这些约束内探索多条合理路径。

HOW DO YOU MEASURE MODEL CAPABILITIES?

Ambiguity

To mirror the work humans actually do, models have to interpret and work with underspecified instructions and messy initial environments.

TAKEAWAYS

- To accurately reflect real-world tasks, models must determine the underlying motivations behind an ask, reason through constraints, and determine success criteria
- Multiple possible approaches and avenues demands more exploration, which requires more advanced reasoning
- The trade-off: standardized evaluation gets much harder

图 5：现实歧义扩大了探索空间，也提高了标准化评估的难度。⁵

2.5 本章小结

环境复杂度无法由时长或工具数量替代。工具协调要求信息跨系统流动；状态变化让行动产生持久后果；顺序复杂度使错误能够沿依赖链级联；任务歧义要求智能体在不完整信息中探索。下一章据此处理评分难题：既要验证最终结果，也要检查抵达结果的过程是否合规。

3 验证结果，也验证路径：开放任务的双重证据

当任务允许多条合理路径并持续改变环境时，单一参考答案已经不足以承担评分责任。最终状态（final state）回答“结果是否成立”，过程轨迹回答“结果怎样获得”。前者关注任务结束后的环境及外部效果，后者记录步骤、工具调用、观察与状态变化。二者联合使用，才能同时识别结果错误、环境副作用和违规捷径（11:01--12:43）。

3.1 从硬可验证领域走向开放输出

硬可验证领域（hard-verifiable domain）允许确定性答案、测试或程序较完整地检查正确性。数学题、数据结构式编程题可以通过证明、固定测试或脚本验证关键结果。确定性验证器（deterministic

⁵ 视频画面时间区间：09:09–10:06。

verifier)基于固定规则给出可复现结果,在检查明确不变量时具有低歧义和高重复性(10:06--10:58)。

经济价值更高的现实任务常包含开放输出与多条合理路径。来源把这类场景称为软可验证领域(soft-verifiable domain):它们难以只靠 Python 脚本、测试用例或形式证明完整判断结果与环境变化是否正确。为此,评估系统会引入评审模型(judge model),字幕也提到有人称其为 critic model(10:43--11:17)。

评审模型可以结合语义与上下文进行判断,确定性验证器则负责稳定检查硬约束。两者是互补机制:开放任务需要更丰富的判断,并不意味着固定测试失去价值。

验证机制的分工

确定性验证器适合回答“某个明确条件是否满足”;评审模型适合综合判断“多个证据是否共同支持任务意图已经实现”。可信评估应先保留可确定检查的不变量,再把开放判断交给有证据访问能力的评审机制。

Verifiers

01 Trajectories are growing **longer** and environments more **complex**

02 From **hard-verifiable domains** (math, Leetcode-style coding) to **soft-verifiable domains** (finance, legal, healthcare, open-ended SWE)

Judge models are a critical part of how we verify correctness and assign reward

图 6: 开放式专业任务从硬可验证领域走向软可验证领域, 评审模型因而成为验证链的一部分。⁶

3.2 最终状态与过程轨迹: 结果证据和手段证据

最终状态能检查任务结束后“世界变成了什么样”。例如, 代码是否通过测试、部署是否恢复、数据库是否处于预期状态。这类证据直接对应交付结果, 也能暴露没有达到目标或造成额外副作用的情况。

过程轨迹则检查智能体如何抵达该状态。它可以揭示读取了哪些信息、调用了哪些工具、改变了哪些对象, 以及关键判断建立在什么观察之上。即使最终输出表面正确, 轨迹仍可能显示智能体使用了禁止手段或依赖了不应获得的信息(11:15--12:43)。

两类证据也各自存在盲区。只看最终状态, 无法证明路径合规; 只看工具调用文本, 也无法确认环境最后是否真的改变。来源后文明确指出, 单看智能体声称调用了什么通常并不可靠, 评审者

⁶ 视频画面时间区间: 10:29-10:58。

还需要读取实际环境状态。因而，本章把“双重证据”视为来源内容的结构化归纳，而非一套来源明示的形式公式。

软件交付类比

最终状态类似验收环境中的运行结果，过程轨迹类似版本历史、CI 日志和操作审计。运行结果通过只能证明当前表象符合条件；审计记录进一步说明变更来源、权限边界和执行顺序是否可信。

Why Rubric-Based Judge Models?

- For entire classes of problems we care about, defining deterministic verifiers is impractical, brittle, or outright impossible
- Not all generated solutions are equal, and not all paths to solutions are equal (e.g. prevent reward hacking)
- Judges look at agent trajectories, not just final outcomes

图 7：评审模型同时检查结果与路径，用于覆盖确定性验证器难以表达的开放式正确性。⁷

3.3 奖励劫持：表面成功为何可能失真

奖励劫持 (reward hacking) 指系统获得高奖励或表面成功，却绕过任务意图、约束或安全边界。来源给出三类风险：智能体逃逸沙箱、读取特权信息、访问编码任务的隐藏测试集 (12:07--12:43)。这些行为可能帮助它构造能通过检查的答案，却破坏了评估真正要测量的能力。

例如，隐藏测试集原本用于检验代码是否具备泛化正确性。如果智能体直接读取测试内容，再针对具体样例修补输出，最终测试结果可能很好，过程却不再代表独立解决问题。最终状态会看到“测试通过”，过程轨迹才可能发现越权读取。反过来，轨迹审计仍需要可信事件记录；若日志可以缺失或篡改，评审模型也无法从不完整证据恢复真实路径。

工程前提：证据自身也要可信

来源提出应检查轨迹，却没有给出事件签名、防篡改存储或完整性协议。工程实现必须额外保证日志可追溯、关键事件不易篡改、工具身份与时间顺序可信。否则，“看过轨迹”无法证明过程合规。

⁷ 视频画面时间区间：11:34--11:58。

3.4 参考答案的有限角色：验证约束，保留路径多样性

开放任务通常存在多个正确解。参考答案或示例轨迹可以帮助评审者理解目标、展示一种可行路径，也可以提供待检查的不变量。它们无法穷举全部合法策略。如果评分器要求实际轨迹与参考轨迹逐步一致，就会人为压缩状态空间（state space），把没有违反目标与约束的替代路径判为错误（12:43--13:28）。

更稳健的做法是分开三类内容：必须达到的最终条件、全过程不得违反的约束、允许自由选择的策略。确定性验证器检查能够程序化表达的条件；评审模型结合最终状态与过程轨迹判断其余语义要求；参考答案只作为解释与校准材料，不能成为唯一合法路线。

开放评估的设计原则

评估对象应是结果不变量与过程约束。参考答案用于说明“一个好解可能是什么样”，不能被当成“所有好解都必须复现的路径”。这种分工既能发现奖励劫持，也能保留合法探索。

3.5 本章小结

开放任务需要双重证据：最终状态验证结果，过程轨迹验证手段。确定性验证器守住可复现的不变量，评审模型处理需要语境的开放判断；奖励劫持案例说明高分或测试通过可能掩盖违规路径；参考答案应帮助理解与校准，不能压缩全部合法状态空间。下一章将进一步追问评审者自身的能力和权限：它必须看懂复杂证据，也必须受到比执行者更严格的变更边界约束。

4 把评审者也当作智能体：工具复用、权限收缩与可查询轨迹

复杂环境中的评审无法只靠阅读一句答案完成。讲者在 13:28--16:33 给出的核心判断是：评审者本身也要具备智能体式的观察、检索与推理能力，同时其权限必须受到更严格的约束。这里的工程目标可以概括为一句话：让评审者看得足够深，改得足够少。

本节核心命题

评审能力需要随任务复杂度扩展，评审权限需要随环境风险收缩。运行框架可以复用观测接口，写入、部署和其他状态变更权限则应单独裁剪。

4.1 “评审者也是智能体”意味着什么

讲者先把讨论从“选择哪个评分模型”推进到“如何为评审者搭建工作环境”。当执行者需要跨日志、代码仓库、持续集成系统和云平台完成任务时，评审者也需要足够的工具来重建事实链。单次模型调用只能处理已经整理好的输入；复杂任务中的证据分散在多个系统、多个时间点和多个状态中，评审者必须主动定位并组合它们。

运行框架已经在前文定义为包围模型的工具接入、上下文管理、生命周期和观测接口。到了评审阶段，其中最关键的能力是**可观测性（observability）**：通过日志、指标、环境状态和查询接口回答“发生了什么”。可观测性提供证据访问，其边界不包含自动变更系统的权力。

审计员类比

执行者像负责排障和上线的工程师；评审者像使用同一组仪表盘的审计员。二者都要理解日志、部署状态和变更结果，审计员的账户只需要观察能力。这个类比解释了为什么工具界面可以共享，而权限配置必须分离。

4.2 部署失败案例：从行动链到验证链

讲者用一次软件部署失败说明评审所需的证据深度（14:17--15:13）。执行者的工作链包含以下环节：

1. 查看代码托管平台的 CI/CD 日志与云端日志，定位失败症状；
2. 根据诊断结果修改代码库；
3. 发起合并请求，并在合并后触发或观察重新部署；
4. 检查部署后的服务状态。

评审者需要验证的对象比“执行者声称完成”更具体。它应重新查看部署后的日志和环境状态，确认故障是否消失、预期服务是否恢复，以及最终状态是否与任务目标一致。仅浏览执行者的工具调用记录容易遗漏外部系统中的真实结果：调用成功不等于部署成功，提交了变更也不等于服务恢复。

这段案例同时承接第三节的双重证据结构。过程轨迹说明执行者查过什么、改过什么、按什么顺序行动；最终状态说明部署后的系统究竟处于何种状态。两类证据互相补足，任何一类单独存在都会留下盲区。

4.3 权限不对称：共享观察接口，收回变更能力

评审者为了查明事实，可以复用执行者使用的日志和状态接口。风险来自权限随接口一并复制：若评审者还能触发部署、修改配置或写入数据库，它在检查期间就可能改变待审对象，导致证据失真，甚至制造新的故障。

讲者提出的具体控制是**只读权限 (read-only permissions)**。评审者能够读取部署日志与运行状态，无法启动新的部署或改变环境。这是一种权限不对称：执行者拥有完成任务所需的变更能力，评审者拥有证明结果所需的观察能力。

评审者的二次变更风险

共享运行框架不代表共享执行权限。若评审者能在检查时改动环境，最终状态可能已经不再对应执行者结束任务时的状态，审计结论也就失去清晰的因果归属。

来源只给出了只读权限这一方向，没有规定账户隔离、临时凭证、字段级脱敏或最小权限策略的实现细节。落地系统仍需明确每个接口的读写语义，并验证所谓“读取”是否会触发刷新、重算或其他隐式状态变化。

Judges Are Agents Too

- Agent harness isn't the only thing that needs to keep up with complex environments: your judge needs to be a capable agent
- Many parts of judge harness design often mirror agent harness design: judges need to use the same tools to explore and verify sources of truth
- Especially true with environments where the state space is both large and critical for correctness

EXAMPLE

Task: Sifting through CI/CD and CloudWatch logs to investigate a broken deployment, fixing the underlying issue, and then opening a PR to redeploy with the fix

Judge: Needs tools to interact with Github and AWS just like the agent harness (with some added safeguards like read-only permissions to prevent accidental state contamination)

图 8: 评审者需要与执行者相称的观测能力, 同时以只读权限避免二次污染环境状态。⁸

4.4 把长轨迹加工成可查询证据

随着任务拉长, 把整段过程轨迹一次性塞入评审模型的上下文窗口会逐渐失效(15:29--16:33)。问题除了长度, 还包括证据结构: 关键故障点可能埋在大量重复日志中, 不同阶段使用相似工具, 单条事件也可能缺少解释其作用的上下文。

讲者建议把轨迹存入数据库, 按语义阶段切分, 为步骤补充元数据, 并在需要时用子代理丰富局部证据。由此形成**可查询轨迹 (queryable trajectory)**: 评审者可以定位关键步骤和候选失败点, 再回读相关上下文, 而非把全部日志当作一个不可分割的文本块。

以部署案例为例, 轨迹可以分为三个阶段: 日志调查、代码修改、变更后验证。每个阶段至少应能关联时间、工具身份、输入对象、输出摘要和环境状态。评审者发现“部署仍然失败”后, 可以反向查询负责重新部署和事后检查的步骤, 再判断问题来自错误诊断、修复无效, 还是部署根本没有发生。

可查询轨迹的教学要点

轨迹加工的目标是让决定性证据可定位、可回溯、可组合。阶段划分和元数据服务于检索, 完整原始证据仍需保留, 以免加工过程把反例或异常路径过滤掉。

⁸ 视频画面时间区间: 13:42-15:13。

Scaling Agent Trajectories

THE BOTTLENECK	Trajectory length scales with task horizon — becoming a bottleneck for judges
	Dumping raw trajectories into a judge's context window no longer works
WHAT WORKS	Trajectories must be queryable , so judges can pull relevant turns, tools, and failure points
	Robust judges should cross-reference trajectory data with environment state

图 9: 长轨迹必须成为可查询证据, 评审者才能定位关键步骤并与环境状态交叉核对。⁹

这一方案依赖若干未在来源中展开的基础设施: 稳定的事件模式、可信时间戳、可识别的工具身份、可靠的阶段边界, 以及不会漏掉决定性事件的索引。它们是实现可查询轨迹的工程前提, 也应成为后续系统审查的一部分。

4.5 本章小结

复杂任务的评审者需要工具、环境观测和多步推理能力。部署失败案例表明, 评审必须重新检查外部系统中的最终状态, 并结合过程轨迹还原变更链。运行框架可以提供共用的观测入口, 只读权限负责限制二次变更风险; 数据库、阶段划分和元数据则把超长记录转化为可查询证据。

评审者已经能够看见并理解证据之后, 下一章要解决评分规则本身的问题: 反馈怎样支持模型学习, 量规怎样保持可执行, 基准分数又应在什么条件下才值得解释。

5 让反馈可学习、让基准可信: 量规工程与四维质量检查

评审者拥有工具和证据以后, 评分仍可能失去训练价值。讲者在 16:33--20:56 把问题归结为两个相互牵制的要求: 反馈要细到能够区分部分进展, 量规也要稳定到评审者能够一致执行。随后, 他用时域、饱和度、广度和奖励粒度四个维度审查现有基准, 说明高分只有放回任务与评分协议中才有意义。

本节核心命题

训练信号的价值由任务分布、证据质量、量规粒度和评审一致性共同决定。粒度不足会掩盖部分能力, 规则过载会使评审者无法稳定应用同一标准。

⁹视频画面时间区间: 15:29–16:33。

5.1 可学习性与奖励信号密度

可学习性 (learnability) 描述任务分布、数据分布和奖励信号能否让训练计算产生有效能力改进。它关注模型从训练经验中能学到什么，其含义不能由某次评测分数直接替代。讲者特别强调，任务选择不当时，大量计算可能消耗在模型无法获得有效学习信号的问题上。

其中一个关键变量是**奖励信号密度 (reward-signal density)**，即系统能在多细的层级上区分有效进展、局部错误和彻底失败。只有终局成败的稀疏反馈会把多种不同失败压成同一个零分；更细的量规可以指出模型在哪个阶段已经掌握能力，在哪个阶段仍需修正。

密度增加也会提高评审负担。尤其面对当前模型尚难可靠完成的前沿任务，标准条目过多、边界过细或依赖关系不清，都会让评审者难以重复地应用量规。这里存在一个操作性权衡：反馈必须提供足够的信息增益，条目数量和复杂度又必须服从可执行性。

“越细越好” 会制造新的噪声

量规条目增加并不自动提升训练质量。若不同评审者对条目边界理解不一，或同一评审者在不同样本上应用标准不稳定，新增信号会变成评分噪声。粒度设计必须与一致性测试同步进行。

5.2 混合验证与条件性部分得分

讲者明确保留了确定性验证器的作用。它可以运行测试、计算指标或生成结构化工件，再由评审模型结合开放式证据判断。这种组合把可复现的不变量检查交给程序，把需要语境和多证据权衡的判断交给评审者。

画面把这种机制称为动态评估时量规 (dynamic evaluation-time rubric)：如果模型在早期步骤犯错，评审可以暂时把该中间结果视为给定条件，继续检查模型是否掌握后续推理。它类似批改多步试题时，保留第一步错误，同时评估后面步骤在条件成立时是否正确。

这类机制提供**部分得分 (partial credit)**，用于分离“前置判断失败”和“后续能力缺失”。它不会把整体错误改判为整体成功。最终任务是否完成仍由既定成功标准决定，分项奖励只说明模型展示了哪些可学习能力。

确定性检查与模型评审的分工

确定性验证器适合检查明确测试、格式、不变量和可重复指标；评审模型适合解释开放输出、多条合法路径和跨系统证据。两者协同时，程序产出的工件也可以成为评审者的输入。

Emerging Rubric-Judge Patterns

Dynamic evaluation-time rubrics

- Even when a model's assumptions, inputs, or early steps were wrong, the judge can grade downstream work as if they were correct, generating a tailored rubric on the fly
- **Example:** in the finance forecasting task, the judge parses out the model's assumptions, calls a tool to plug them into a golden financial model, and uses the financial model's outputs to check for partial credit

Combining deterministic and judge-based verifiers

- **Piecewise:** independent, weighted components — e.g. test cases catch regressions in a code refactor, while a judge separately evaluates whether the changes were meaningful
- **Deterministic artifact for judge review:** precompute objective metrics, then hand them to the judge to interpret — e.g. exposing hardware-dependent performance metrics as a CSV for the judge to grade

图 10: 动态量规与混合验证把部分得分、确定性指标和评审判断组合成更密集的学习信号。¹⁰

5.3 量规质量保障：先检查评分规则

当人工智能参与量规创建、验证或专家辅助时，量规自身也需要更严格的测试。讲者把**量规质量保障**（**rubric quality assurance**）提到与任务增长同等重要的位置，并明确强调覆盖度与专家一致性。

覆盖度回答：量规是否涵盖任务成功所需的关键结果、过程约束和典型失败。专家一致性回答：有资质的评审者面对同一证据时，能否稳定地给出相近判断。二者分别控制“漏掉重要标准”和“标准无法稳定执行”这两类风险。

已确认的画面列出五项量规评审测试：Gold、No-Op、Variance、Coverage 和 Expert Agreement。其中，Gold 检查金标准案例应获得高于 98% 的分数；No-Op 检查无实质工作的输出应低于 3%；Variance 检查重复评审的方差应低于 2%；Coverage 要求完整覆盖从初始状态到金标准的差异；Expert Agreement 则检查专家是否认可评分结果。这些是来源幻灯片的量规 QA 清单，并非适用于所有系统的通用阈值。

¹⁰视频画面时间区间：17:17–17:53。

Rubric-Judge QA

For each rubric we produce, we run 5 different tests:

- Gold: We grade a verified “golden solution,” ensuring every attempt scores >98%
- No-Op: We grade the initial starting stage, ensuring every attempt scores <3%
- Variance: Same attempt, same judge, repeated multiple times. We minimize variance at both the criteria and sub-criteria level. For each rubric, we keep variance <2% for the same attempt
- Coverage: % of differences covered by the rubric criteria. We ensure our rubric covers 100% of the differences in between the initial and gold states with no gaps
- Expert Agreement: We have an expert and an LLM-as-a-judge independently grade the task. Then, we have the expert examine disagreements and determine any relevant improvements to the rubric

图 11: 量规质量保障同时检查金标准、空操作、重复方差、覆盖度和专家一致性。¹¹

5.4 基准的四维质量检查

在收束讨论中，讲者批评了一组金融相关训练与评测数据，并给出四个可迁移的审查维度。幻灯片以 GDPval、Banker Tool Bench 和 APEX Agents 为例，展示它们的发布方、平均人类耗时、评分指标、模型和得分。这些数值是讲者幻灯片对所示版本的报告，其中的横向可比性仍取决于任务分布与指标协议。

1. **时域**：任务的人类完成时间是否达到声明的能力前沿，并且是否带有清晰的成功阈值和测量协议。较短任务被冠以“长周期”标签，会夸大基准测量的能力跨度。
2. **饱和度**：基准饱和度 (benchmark saturation) 用于判断当前模型是否已经解决较大比例的任务，以及该基准还剩多少区分前沿能力的空间。具体比例必须绑定版本、子集和指标语义。
3. **广度**：基准广度 (benchmark breadth) 检查任务类型、职业活动和业务领域的覆盖范围。题目数量多仍可能集中在狭窄工作切片，因而无法代表整个领域。
4. **奖励粒度**：评分是否能区分关键中间能力和部分进展，并且评审者能否可靠执行这些细则。

幻灯片报告三个基准的平均人类耗时分别为 7、5 和 1.4 小时，评分指标分别为 Win Rate、Score@3 和 Pass@1，展示得分为 84.9%、58.1% 和 57%。这些来源图表值用于说明“四维联合检查”的必要性：某些任务集可能集中于电子表格或投资银行工作，信用、债务和风险等领域覆盖不足；较高得分也可能表明前沿区分空间已经收窄。

¹¹ 视频画面时间区间：17:59–18:24。

Existing literature suggests a lot of the data being used to train and evaluate models right now is flawed

	GDPval	Banker Tool Bench	APEX Agents
Published by	OpenAI	Handshake AI	Mercor
Average Human Hours Per Task	7 hours	5 hours	1.4 hours
Score Metric	Win Rate	Score@3	Pass@1
Model Used	GPT 5.5	GPT 5.4	Gemini 3.5 Flash
Score	84.9%	58.1%	57%

图 12: 来源幻灯片以时域、评分指标、模型和得分对比三个金融任务基准。¹²

5.5 讲者的收束案例及其证据边界

讲者随后以 Theta Software 的金融任务数据作对照。来源幻灯片报告人类完成一项任务平均用时为 14.7 小时；讲者口述将其概括为约 15 小时，并明确该估计来自 50 项任务样本。幻灯片还报告：GPT-5.5 每项任务的模型运行时间为 92.4 分钟，每项任务平均包含 3,600 个单元格变更，每个 Excel 模型平均包含 5.8 张工作表；成果表使用的指标名称是 Mean@5。这些数字是 Theta 报告的专有数据结果，演讲没有提供独立外部审计。

这段案例的教学价值仍然清楚：一个基准若要支持“长周期能力”结论，需要同时证明任务确实足够长、当前模型尚未饱和、业务覆盖足够广，以及量规能给出有用的细粒度反馈。任何一项缺失，单个排行榜分数都会产生解释歧义。

At Theta, we build and think deeply about long horizon environments for frontier models—the right way

Metric	Average
Human time to complete one task	14.7 hr
Model runtime per task	92.4 min
Cell changes per task	3,600 cells
Sheets per Excel model	5.8

* Runtime figures reported for GPT-5.5.

Subdomain	Mean@5			
	GPT-5.5	Opus 4.8	DeepSeek-V4-Pro	MiniMax M3
Corporate Finance	57.94%	54.21%	42.72%	31.12%
Investment Banking	68.09%	71.62%	33.69%	48.48%
Private Equity	48.41%	45.45%	43.56%	36.42%
Credit	49.37%	46.03%	39.59%	33.27%
Debt	46.12%	44.54%	29.91%	27.21%
Real Estate	54.13%	56.23%	27.37%	35.73%
Risk	62.71%	63.47%	43.17%	39.50%

图 13: Theta 展示的金融任务覆盖多个子领域，并用平均任务统计与 Mean@5 结果说明其环境仍具有挑战性。¹³

¹² 视频画面时间区间：18:24–18:51。

¹³ 视频画面时间区间：20:29–20:56。

5.6 本章小结

可学习性要求训练任务、数据分布和奖励信号共同提供可用反馈。奖励信号过稀会掩盖部分进展，量规过密又会压低评审一致性；混合验证和条件性部分得分提供了拆解能力信号的路径。量规本身还需接受覆盖度与专家一致性检查。

讲者的基准批评进一步表明，时域、饱和度、广度和奖励粒度必须联合审查。下一章将把这些判断与前文的因果状态、双重证据和权限边界连接成一条可执行的环境审查链。

6 总结与延伸：用闭环审查长周期环境

这场演讲的实质性结论可以压缩为：可信的长周期训练与评测依赖任务、环境、验证器、评审者、量规和质量保障的共同设计。长时间运行和醒目的排行榜分数只提供局部现象；只有当早期决策真实影响后续状态、证据能够还原结果与路径、评审权限受到约束、奖励规则经过质量检查时，这些现象才具有稳定的能力解释。

6.1 讲者的收束判断：先检查数据能测量什么

在 18:24--20:56 的结尾讨论中，讲者回到开篇对“长周期”的定义，并指出现有训练和评测数据可能存在四类缺陷：任务时域低于所声称的能力前沿，基准已经出现一定饱和，业务范围集中在狭窄切片，奖励信号又不足以支持细粒度学习。

讲者用金融任务集说明这四类风险可能同时出现。某些数据集中在电子表格或投资银行活动，无法覆盖信用、债务、风险等更广业务；有些任务对当前模型的区分空间已经缩小；过于粗糙的评分又难以指出模型具体学会了什么。Theta Software 的对照案例意在强调更长的人类任务跨度、更广的领域分布、较长的模型工作过程和仍然存在的难度；第 13 图保留了讲者用于这一结论的来源幻灯片。

讲者结论的可迁移含义

基准标签不能替代测量协议。解释一个“长周期”成绩之前，应先确认任务跨度、前沿区分力、业务覆盖和奖励粒度是否共同支持该解释。

例行致谢位于 20:53--20:59，没有增加教学信息，因此未纳入总结。

6.2 跨节综合：六个环节怎样互相约束

以下闭环由编辑依据讲者各段论述综合整理，并非讲者在视频中发布的正式框架。

第一节建立测量坐标系，说明任务时域、人类估时和模型原生轨迹指标都会受到协议与实现噪声影响。这个结论直接约束第五节：基准分数必须连同阈值、模型、运行框架、任务分布和评分协议一起报告，跨基准比较才能获得可解释性。

第二节讨论状态变化和顺序复杂度。早期查询、误读或变更会限制后续行动，因此第三节需要同时检查最终状态与过程轨迹。最终状态能显示结果，过程轨迹能暴露错误如何级联、约束是否被绕过，以及表面成功是否来自奖励劫持。

第四节进一步说明，能够理解复杂证据的评审者也需要工具与运行框架。它的能力边界与权限边界必须分别设计：可观测性支持调查，只读权限维持待审状态的因果归属。第五节的量规则把这些证据转化为奖励；量规质量保障检查转换过程是否覆盖充分、执行一致。

闭环中的三组互补关系

任务时域与因果深度分别描述跨度和依赖；最终状态与过程轨迹分别证明结果和路径；评审能力与评审权限分别决定能看懂多少证据和能改变多少环境。每组概念都需要同时保留。

6.3 面向实践的七步审查链

下面的顺序同样属于编辑性综合，目的是把演讲内容转成可以执行的设计检查。

1. **定义成功阈值。**写明什么状态算完成、成功概率如何统计、人类完成时间如何估计，以及运行框架和模型版本。
2. **确认真实状态依赖。**追踪早期行动是否会持续改变环境，并检查后续步骤是否真正依赖这些变化；串联无关任务不能充当因果深度的证据。
3. **规定双重证据。**列出必须检查的最终状态、不变量和外部效果，同时保存步骤、工具调用、观察与状态变更形成的过程轨迹。
4. **配置权限受限的评审工具。**让评审者访问必要日志、指标和状态接口，并用只读权限隔离部署、写库和配置修改能力。
5. **建立可查询轨迹。**保存原始事件，按阶段组织证据，附加可信时间、工具身份和对象元数据，使关键步骤与失败点能够被定位和回溯。
6. **校验量规质量。**检查标准是否覆盖关键结果与违规路径，专家是否能一致应用；需要部分得分时，明确分项能力与总体成功的边界。
7. **最后解释基准分数。**同时报告任务时域、饱和度、业务广度和奖励粒度，再判断成绩究竟代表能力增长、任务变窄，还是测量协议变化。

审查链中的断点

任一环节失真都会向后传播：任务缺少状态依赖会让长轨迹失去能力含义；轨迹不完整会让违规路径无法发现；评审者权限过大会污染最终状态；量规未经测试会把证据转换成不稳定奖励。

6.4 局限与开放问题

演讲提供了方向和案例，仍留下若干需要工程与研究共同回答的问题。

- **轨迹完整性：**怎样证明日志没有遗漏、重排或被执行者影响？来源没有给出事件签名、防篡改存储或跨系统时钟校准方案。
- **合法解法多样性：**怎样测量一个开放任务的合理路径范围，并让评审者接受新颖解法，同时拒绝违反安全边界的捷径？
- **评审一致性：**怎样在不同模型、不同提示和不同证据顺序下测试评审结果的稳定性？专家一致性提供参照，具体协议仍待定义。
- **奖励粒度：**怎样找到足以支持学习、又不会让评审规则过载的条目层级？这一选择依赖任务分布、失败模式和评审能力，演讲没有给出统一阈值。

- **权限语义：**只读接口是否真的没有副作用，云平台中的查询、刷新和诊断动作如何分类，仍需逐系统验证。
- **基准外推：**金融任务案例能否代表其他职业领域，需要更广任务分布和独立复现支持。

这些未知项不会削弱演讲的中心方向；它们界定了把原则变成可靠基础设施时必须补齐的证据。

6.5 本章小结

整篇内容最终落在一条可迁移的判断上：先审查环境与验证闭环，再解释“长”和“高分”。任务跨度必须带阈值和协议，长任务必须包含真实因果依赖，验证必须覆盖最终状态与过程轨迹，评审者必须具备足够可观测性并受到只读权限约束，奖励量规还必须兼顾可学习性与一致性。

当这条链闭合时，长时间运行能够对应持续的自主能力，基准分数也能够对应清晰的成功含义；当链条存在断点时，时间、token 或排行榜成绩都只能作为待解释的表面信号。